INSTITUTO PROFESIONAL SANTO TOMÁS

Ingeniería en Informática

Gestión de Proyectos y Equipos de Trabajo

Evaluación N°3

DIAGRAMAS DE SECUENCIA Y CASOS DE USO DE LOS FLUJOS DEL SISTEMA

Proyecto: Sistema de seguimiento y fidelización de tratamientos

Cliente: Centro Médico Vida Norte SpA

Equipo: ARM Systems

Integrantes:

Mauricio Aliaga

Nicolás González

Fabián Moore

Lukas Rozas

Docente: Pedro Fernández Urrutia

Arica, noviembre de 2026

ÍNDICE

1. Introducción

1.1 Propósito

1.2 Alcance

1.3 Cómo leer este documento

1.4 Fuentes

2. Visión general del sistema

2.1 Arquitectura

2.2 Actores

2.3 Modelo de datos

2.4 Casos de uso

2.5 Reglas de negocio

2.6 Cobertura del alcance del proyecto

3. Flujos del sistema

3.1 F-01: Iniciar sesión

3.2 F-02: Verificar la sesión al abrir una pantalla

3.3 F-03: Cerrar sesión

3.4 F-04: Consultar mis pacientes y su hilo

3.5 F-05: Enviar una recomendación a un paciente

3.6 F-06: Ver el estado de lectura de lo enviado

3.7 F-07: Ver el resumen de lectura

3.8 F-08: Ver mis recomendaciones

3.9 F-09: Marcar una recomendación como leída

3.10 F-10: Autorizar cada petición a la API

3.11 F-11: Aplicar migraciones y cargar datos de demostración

4. Manejo de errores

5. Limitaciones conocidas

6. Evidencia del incremento

6.1 Módulos funcionales entregados

6.2 Pruebas realizadas

6.3 Constancia de revisión y aprobación

Anexo A. Rutas de la API

Anexo B. Mapa del código

1. Introducción

1.1 Propósito

Este documento describe, flujo por flujo, cómo funciona el primer avance del Sistema de seguimiento y fidelización de tratamientos del Centro Médico Vida Norte SpA. Para cada flujo presenta una descripción, un diagrama de secuencia y la especificación del caso de uso correspondiente. Sirve como documentación para quien deba mantener el sistema, como apoyo para las pruebas de aceptación con el cliente y como evidencia del incremento de software entregado.

1.2 Alcance

Cubre lo que el sistema hace hoy: inicio y cierre de sesión con dos roles (doctor y paciente), envío de recomendaciones del doctor a sus pacientes, lectura y confirmación de lectura por parte del paciente, un resumen de lectura para el doctor y la preparación de la base de datos. No cubre las funciones que el proyecto define y que todavía no se construyen, como las reservas de horas, el formulario estandarizado de contacto, las notificaciones, el historial de accesos y la Clave Única simulada. La sección 2.6 indica el estado de cada requisito.

1.3 Cómo leer este documento

Los diagramas de secuencia usan la notación UML. Los actores se dibujan como figuras y los componentes como rectángulos, y el tiempo avanza hacia abajo. Una flecha continua es una llamada y una flecha punteada es una respuesta. Los marcos alt (alternativa), opt (opcional) y loop (repetición) agrupan los mensajes que dependen de una condición.

Cada diagrama muestra el camino principal y la alternativa más relevante. Todas las alternativas se detallan en la tabla del caso de uso. Los nombres de clases, métodos y rutas son los que existen en el código. Las reglas de negocio se identifican como RN-01 a RN-11 y se definen en la sección 2.5.

1.4 Fuentes

El código fuente del sistema (carpeta arm-seguimiento, con el backend y el frontend) y los documentos de gestión del proyecto: caso de negocio, plan de gestión de beneficios, acta de constitución, plan de gestión del alcance, registro de interesados y plan de gestión del equipo.

2. Visión general del sistema

2.1 Arquitectura

El sistema tiene tres partes. El navegador ejecuta la aplicación Angular 22, que usa Angular Material para los componentes de la interfaz y Chart.js para los gráficos. La API es una aplicación NestJS 12 que expone rutas REST bajo /api, protegidas por guardias globales de autenticación y de roles. Los datos están en MariaDB 12 y se acceden con TypeORM 1.1. En desarrollo, el servidor de Angular reenvía las llamadas a /api hacia la API mediante un proxy, de modo que el navegador habla con un solo origen.

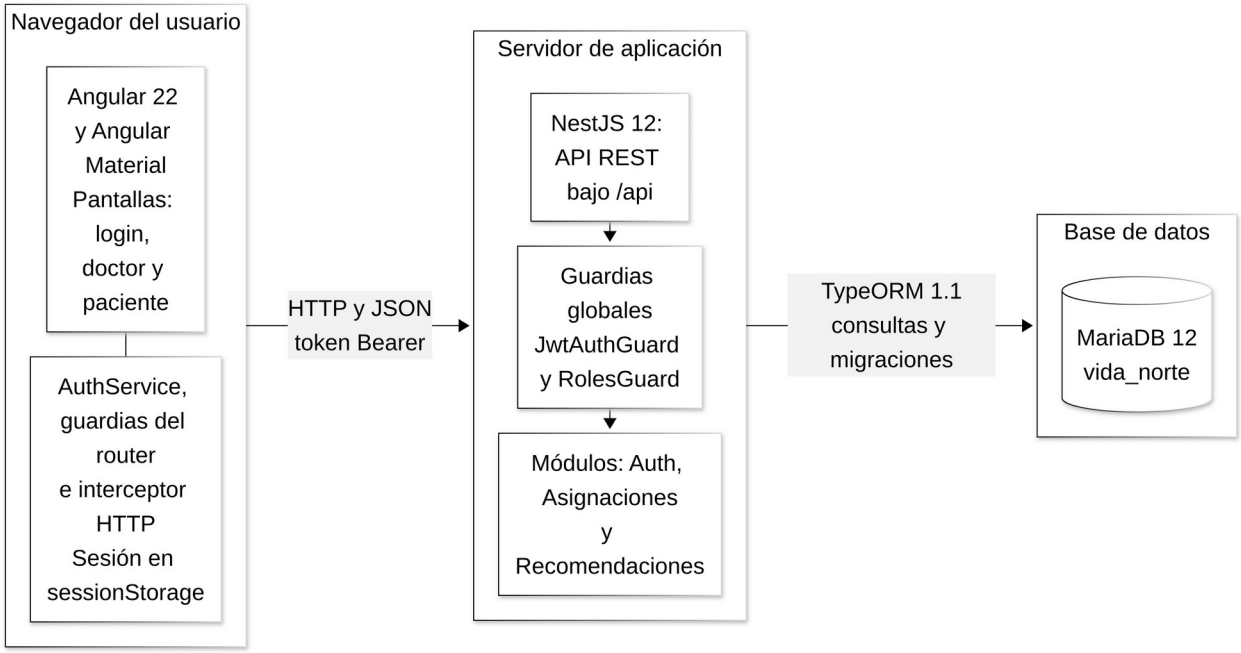


Figura 1. Arquitectura del sistema.

2.2 Actores

Actor	Descripción	Qué hace en el sistema
Doctor	Profesional de salud del centro que tiene pacientes asignados.	Ve sus pacientes, envía recomendaciones, revisa si fueron leídas y consulta el resumen.
Paciente	Persona atendida por uno o más doctores.	Ve las recomendaciones que recibió y las marca como leídas.
Responsable técnico	Integrante del equipo que prepara el sistema.	Aplica las migraciones y carga los datos de demostración.
Sistema	Componentes que actúan sin intervención del usuario.	Verifica la sesión, autoriza cada petición y actualiza las pantallas cada 20 segundos.

2.3 Modelo de datos

La base de datos vida_norte tiene tres tablas. La tabla usuarios guarda a doctores y pacientes, distinguidos por el campo rol, y la contraseña se guarda solo como hash bcrypt. La tabla doctor_paciente vincula a cada doctor con sus pacientes, con una clave primaria compuesta. La tabla recomendaciones guarda cada indicación con su doctor, su paciente, su tipo, su texto y las fechas de envío y de lectura. Las claves foráneas impiden eliminar a un usuario que tenga recomendaciones.

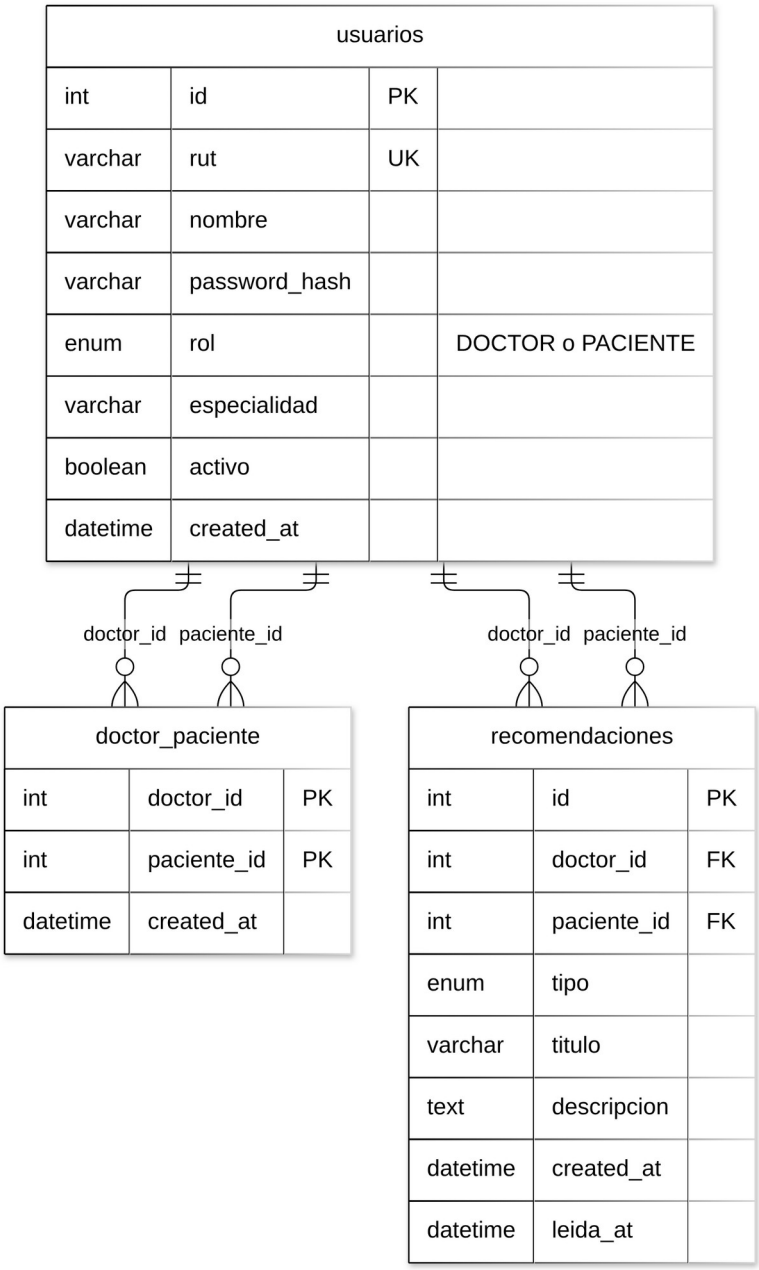


Figura 2. Modelo de datos.

Tabla	Campo	Descripción
usuarios	id	Identificador numérico, clave primaria.
usuarios	rut	RUT normalizado (12345678-5). Es único.
usuarios	nombre	Nombre completo, tal como se muestra en pantalla.
usuarios	password_hash	Hash bcrypt de la contraseña. Nunca se devuelve por la API.
usuarios	rol	DOCTOR o PACIENTE.

Tabla	Campo	Descripción
usuarios	especialidad	Especialidad del doctor. Es nula para los pacientes.
usuarios	activo	Indica si el usuario puede ingresar.
usuarios	created_at	Fecha de creación, en UTC.
doctor_paciente	doctor_id, paciente_id	Clave primaria compuesta. Ambas son claves foráneas a usuarios.
doctor_paciente	created_at	Fecha de la asignación, en UTC.
recomendaciones	id	Identificador numérico, clave primaria.
recomendaciones	doctor_id, paciente_id	Quién la envió y a quién. Claves foráneas a usuarios.
recomendaciones	tipo	MEDICAMENTO, RUTINA, HABITO_EVITAR u OTRO.
recomendaciones	titulo, descripcion	Título (hasta 150 caracteres) e indicaciones (hasta 2000).
recomendaciones	created_at	Fecha y hora de envío, en UTC.
recomendaciones	leida_at	Fecha y hora en que el paciente la leyó. Es nula mientras no se lee.

2.4 Casos de uso

La figura siguiente muestra los casos de uso a los que accede cada actor. El caso CU-02 (verificar la sesión) y el caso CU-10 (autorizar cada petición) son comportamientos del sistema que forman parte de todos los casos protegidos, por eso aparecen como casos incluidos. El caso CU-11 lo ejecuta el responsable técnico fuera de la aplicación y no se dibuja.

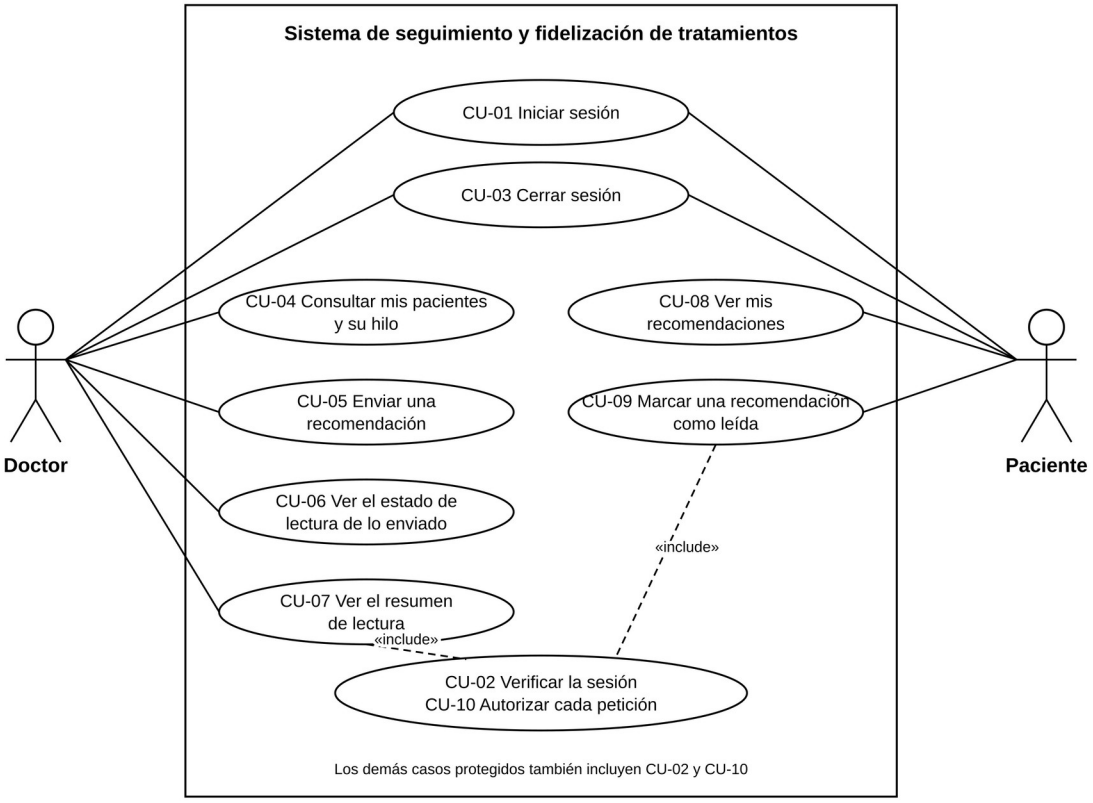


Figura 3. Casos de uso del sistema.

Código	Caso de uso	Actor	Interfaz
CU-01	Iniciar sesión	Doctor o Paciente	POST /api/auth/login (público)
CU-02	Verificar la sesión al abrir una pantalla	Doctor o Paciente	GET /api/auth/me
CU-03	Cerrar sesión	Doctor o Paciente	Ninguna.
CU-04	Consultar mis pacientes y su hilo	Doctor	GET /api/doctor/pacientes. GET /api/recomendaciones/emitidas.
CU-05	Enviar una recomendación a un paciente	Doctor	POST /api/recomendaciones
CU-06	Ver el estado de lectura de lo enviado	Doctor	GET /api/doctor/pacientes. GET /api/recomendaciones/emitidas.
CU-07	Ver el resumen de lectura	Doctor	Ninguna propia. Usa los datos de los flujos F-04 y F-06.
CU-08	Ver mis recomendaciones	Paciente	GET /api/recomendaciones/mias
CU-09	Marcar una recomendación como leída	Paciente	PATCH /api/recomendaciones/:id/leida

Código	Caso de uso	Actor	Interfaz
CU-10	Autorizar cada petición a la API	Sistema	Todas las rutas de /api.
CU-11	Aplicar migraciones y cargar datos de demostración	Responsable técnico del equipo	Línea de comandos: migration:run, migration:revert, migration:show, seed y seed:reset.

2.5 Reglas de negocio

Las reglas siguientes se cumplen en todos los flujos y se citan por su código en cada caso de uso.

Código	Regla
RN-01	El RUT se valida con el módulo 11 (7 u 8 dígitos y dígito verificador) en el navegador y en el servidor. Se guarda normalizado, sin puntos y con guion (por ejemplo, 12345678-5).
RN-02	La sesión dura 8 horas, que es la vigencia del token, y pertenece a la pestaña (sessionStorage).
RN-03	Toda ruta de la API exige sesión, salvo el login. Cada operación exige el rol que le corresponde: doctor o paciente.
RN-04	Un doctor solo ve y solo escribe a los pacientes asignados a él en la tabla doctor_paciente. Un paciente puede tener varios doctores.
RN-05	Una recomendación pertenece a un doctor y a un paciente. En este avance no se edita ni se elimina. El paciente solo accede a las suyas.
RN-06	Una recomendación está sin leer (leida_at nulo) o leída (leida_at con fecha). Marcarla es idempotente y no se deshace.
RN-07	Las fechas se guardan en UTC y se muestran en la hora local del navegador.
RN-08	El título tiene entre 3 y 150 caracteres y las indicaciones entre 5 y 2000. Se recortan los extremos y se eliminan los caracteres de control. Los tipos permitidos son MEDICAMENTO, RUTINA, HABITO_EVITAR y OTRO.
RN-09	La base de datos decide quién tiene acceso: un usuario inactivo o inexistente pierde el acceso en la siguiente petición, aunque su token no haya vencido.
RN-10	El login no revela por qué falló: responde igual cuando el RUT no existe, el usuario está inactivo o la contraseña es incorrecta.
RN-11	El servidor no revela la existencia de datos ajenos: responde igual ante un paciente no asignado o inexistente (403) y ante una recomendación ajena o inexistente (404).

2.6 Cobertura del alcance del proyecto

La tabla compara los requisitos de alto nivel del acta de constitución con lo que este avance incluye.

Requisito de alto nivel	Estado	Detalle
Reserva de horas médicas (por el paciente y por recepción)	No incluido	Queda para una etapa posterior.
Formulario estandarizado de contacto (llamada, WhatsApp y correo)	No incluido	Queda para una etapa posterior.
Seguimiento de tratamientos: recordatorios y sugerencias del médico	Parcial	Las sugerencias del médico funcionan con lectura registrada. Faltan los recordatorios por WhatsApp y correo.
Trazabilidad de acciones, responsable y estado de cada hilo	Parcial	Se registra quién envió, cuándo y si fue leída. Faltan los estados abierto, finalizado y desertor.
Dashboard de continuidad para médicos	Parcial	Resumen de lectura con datos reales. Faltan los indicadores de continuidad y de desertores.
Perfiles por rol, historial de accesos y máxima seguridad posible	Parcial	Perfiles por rol y autorización en cada petición. Falta el historial de accesos.
Uso opcional para el paciente y contacto no invasivo	Parcial	El paciente solo recibe y confirma lectura. No hay contactos automáticos.
Inicio de sesión con Clave Única mediante simulación	No incluido	El avance usa RUT y contraseña propios del sistema.

3. Flujos del sistema

Cada flujo se presenta con su descripción, su diagrama de secuencia y la especificación de su caso de uso. El identificador del flujo (F-01 a F-11) coincide con el del caso de uso (CU-01 a CU-11).

3.1 F-01: Iniciar sesión

Este flujo permite que un doctor o un paciente ingrese al sistema con su RUT y su contraseña. Es la única pantalla pública: el resto de las rutas exige una sesión válida.

La pantalla da formato al RUT mientras se escribe (123456785 pasa a 12.345.678-5) y comprueba el dígito verificador con el algoritmo de módulo 11 antes de enviar algo al

servidor. El servidor repite la validación, porque no corresponde confiar en lo que llega desde el navegador.

Si las credenciales son correctas, la API firma un token JWT con el identificador, el rol y el nombre del usuario, con vigencia de ocho horas. La pantalla lo guarda en sessionStorage, de modo que la sesión pertenece a la pestaña: puede haber un doctor en una pestaña y un paciente en otra, y la sesión termina al cerrar la pestaña. Luego el usuario va a /doctor o a /paciente según su rol, y la pantalla de login se reemplaza en el historial para que el botón Atrás no regrese a ella.

El servidor responde lo mismo cuando el RUT no existe, el usuario está inactivo o la contraseña es incorrecta («RUT o contraseña incorrectos»). Cuando el usuario no existe compara igual la contraseña contra un hash falso, para que el tiempo de respuesta no revele qué RUT están registrados.

Diagrama de secuencia

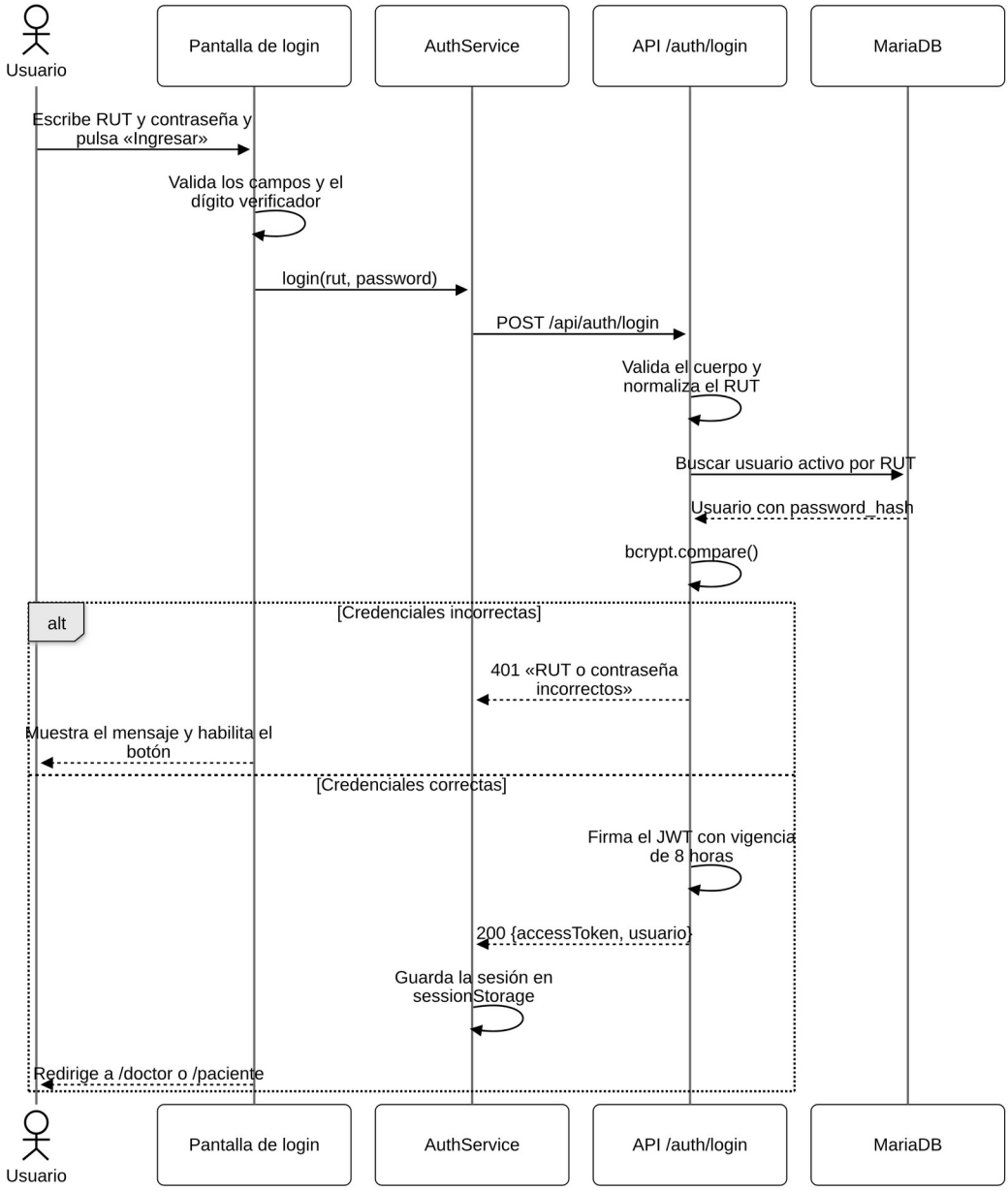


Figura 4. Secuencia del flujo F-01, Iniciar sesión (camino principal y credenciales incorrectas).

Caso de uso

Caso de uso CU-01: Iniciar sesión	
Actor principal	Doctor o Paciente
Actores secundarios	API de autenticación y base de datos

Descripción	El usuario ingresa con su RUT y su contraseña y obtiene una sesión asociada a su rol.
Precondiciones	El usuario existe y está activo en la tabla usuarios. No tiene una sesión abierta en la pestaña. Si la tiene, /login lo envía a su pantalla.
Disparador	El usuario abre /login.
Flujo principal	1. El usuario escribe su RUT. La pantalla lo formatea mientras escribe. 2. Escribe su contraseña. Puede mostrarla con el ícono del ojo. 3. Pulsa «Ingresar». 4. La pantalla comprueba que ambos campos tengan valor y que el RUT tenga el dígito verificador correcto. 5. AuthService envía POST /api/auth/login con el RUT y la contraseña. 6. La API valida el cuerpo de la solicitud y normaliza el RUT. 7. Busca al usuario activo con ese RUT y compara la contraseña con bcrypt. 8. Firma el token y responde con el token y los datos del usuario. 9. AuthService guarda la sesión en sessionStorage. 10. La pantalla redirige a /doctor o a /paciente.
Flujos alternativos	A1. Formulario inválido (paso 4): se muestran los mensajes junto a los campos y no se envía la solicitud. A2. RUT con dígito verificador incorrecto en el servidor: responde 400 con «El RUT ingresado no es válido». A3. Credenciales incorrectas: responde 401 con «RUT o contraseña incorrectos». El formulario conserva el RUT y el botón se habilita de nuevo. A4. Servidor no disponible: se muestra «No se pudo conectar con el servidor. Revise su conexión e intente de nuevo en unos minutos.» A5. Doble clic o Enter repetido: mientras la primera solicitud está en curso se ignoran las siguientes.
Postcondiciones	La sesión queda en sessionStorage de la pestaña. El usuario está en la pantalla de su rol.
Reglas de negocio	RN-01, RN-02, RN-09, RN-10
Requisito relacionado	Perfiles por rol y máxima seguridad posible (Acta de constitución, objetivo OE4). Cubre los perfiles por rol; no cubre el historial de accesos ni la Clave Única simulada.
Interfaz	POST /api/auth/login (público)
Implementación	frontend: login/login.ts, core/auth.service.ts, core/rut.ts. backend: auth/auth.controller.ts, auth/auth.service.ts, auth/login.dto.ts, common/rut.util.ts.

3.2 F-02: Verificar la sesión al abrir una pantalla

Cada vez que se carga la aplicación o se escribe una dirección protegida, el router ejecuta una guardia antes de mostrar la pantalla. La guardia no se fía de lo que está guardado

en el navegador, porque el rol y el nombre guardados se pueden modificar desde las herramientas del navegador. Por eso, una vez por carga de página, AuthService consulta GET /api/auth/me. La API responde con el rol y el nombre que figuran en la base de datos y el cliente corrige lo guardado si difiere.

Con esto, un paciente que edite su rol a DOCTOR en el navegador es devuelto a /paciente, y una sesión con token vencido, firma falsa o usuario desactivado se cierra y vuelve a /login.

Si no hay conexión con el servidor, la guardia deja pasar con los datos guardados. Las peticiones siguientes igualmente fallan si el usuario no tiene permiso, porque la autorización real ocurre siempre en la API.

Diagrama de secuencia

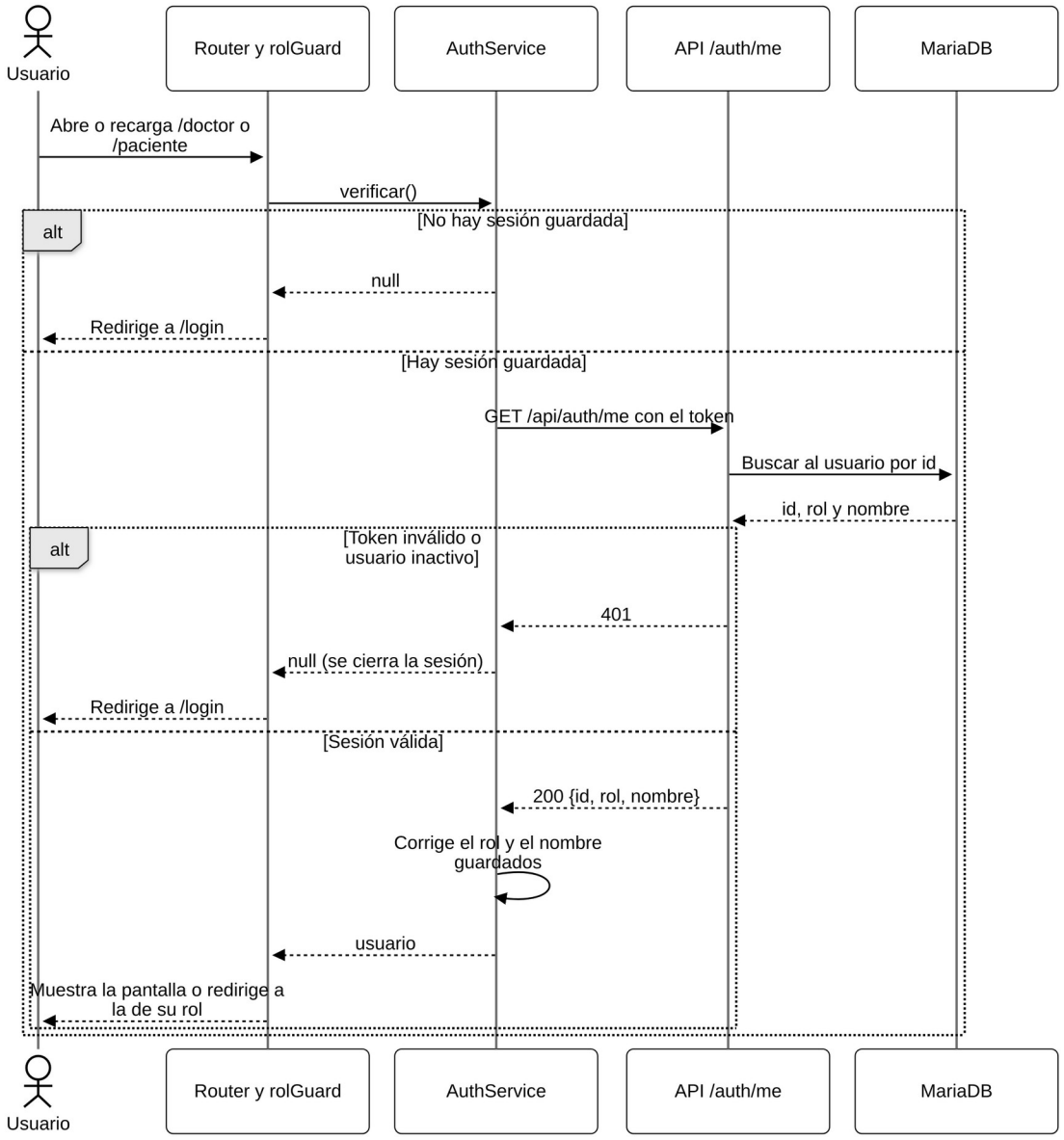


Figura 5. Secuencia del flujo F-02, Verificar la sesión al abrir o recargar una pantalla.

Caso de uso

Caso de uso CU-02: Verificar la sesión al abrir una pantalla	
Actor principal	Doctor o Paciente (el flujo se dispara al navegar)
Actores secundarios	Router, AuthService, API y base de datos
Descripción	Confirmar con el servidor quién es el usuario y decidir a qué pantalla corresponde.

Precondiciones	Ninguna. El flujo también se ejecuta cuando no hay sesión.
Disparador	El usuario abre o recarga /doctor o /paciente, o escribe esa dirección en el navegador.
Flujo principal	1. El router activa rolGuard con el rol que exige la ruta. 2. rolGuard pide a AuthService que verifique la sesión. 3. AuthService lee la sesión de sessionStorage. La descarta si el JSON está dañado o el token venció. 4. Si la sesión ya se verificó en esta carga de página, devuelve el usuario guardado. 5. Si no, envía GET /api/auth/me con el token. 6. La API valida el token y consulta al usuario en la base de datos. 7. AuthService corrige el rol y el nombre guardados si difieren de los del servidor. 8. Si el rol coincide con el de la pantalla, se muestra. Si no, se redirige a la pantalla del rol real.
Flujos alternativos	A1. Sin sesión: redirige a /login. A2. Token inválido, vencido o usuario inactivo (401): el interceptor cierra la sesión y redirige a /login. A3. Rol distinto al de la pantalla: redirige a la pantalla del rol real. A4. Sin conexión con el servidor: se muestra la pantalla con los datos guardados. A5. Si hay sesión y se abre /login, loginGuard redirige a la pantalla del rol sin consultar al servidor.
Postcondiciones	La sesión guardada coincide con la base de datos. El usuario ve solo la pantalla de su rol.
Reglas de negocio	RN-03, RN-09
Requisito relacionado	Perfiles por rol (OE4).
Interfaz	GET /api/auth/me
Implementación	frontend: core/guards.ts, core/auth.service.ts, core/auth.interceptor.ts. backend: auth/jwt-auth.guard.ts, auth/auth.controller.ts.

3.3 F-03: Cerrar sesión

El usuario cierra la sesión con el botón «Cerrar sesión» del encabezado. AuthService borra la sesión de la memoria y de sessionStorage y navega a /login reemplazando la entrada actual del historial, de modo que el botón Atrás no vuelve a la pantalla protegida.

El cierre es solo local: la API no mantiene sesiones y no hay una llamada al servidor. El token sigue siendo válido hasta que venza, a las ocho horas, o hasta que el usuario sea

desactivado. Es una limitación conocida de los tokens JWT sin lista de revocación (ver el capítulo 5).

Diagrama de secuencia

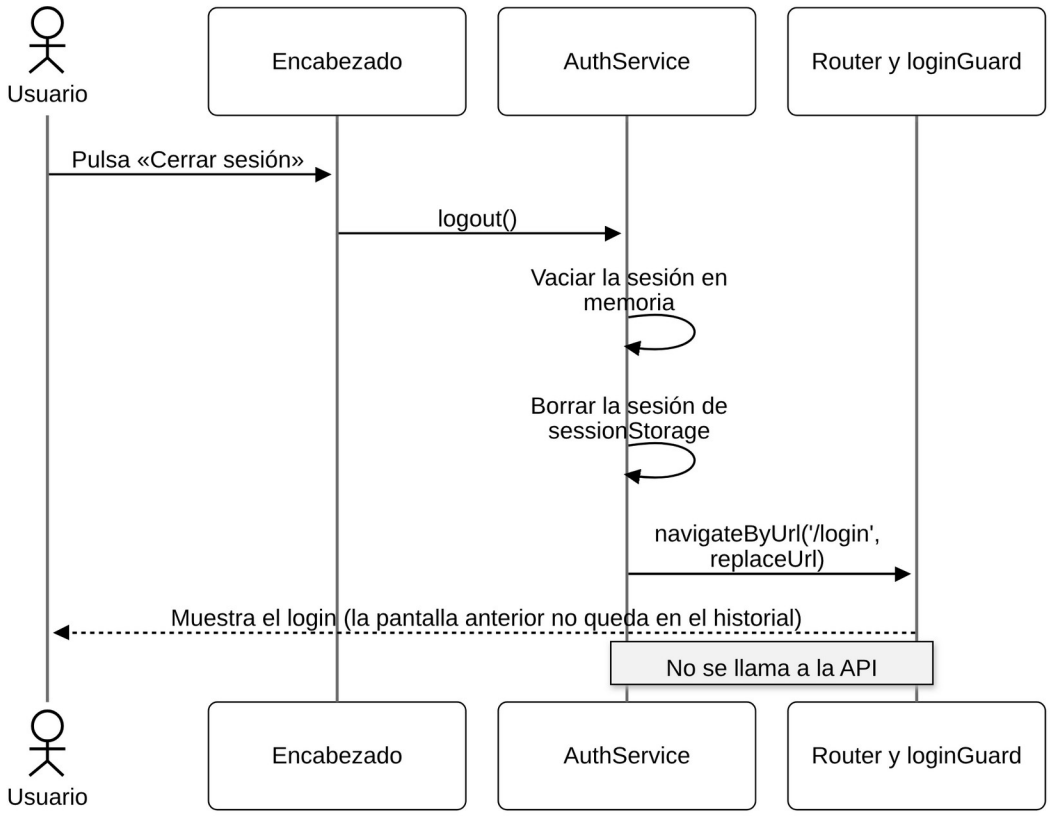


Figura 6. Secuencia del flujo F-03, Cerrar sesión.

Caso de uso

Caso de uso CU-03: Cerrar sesión	
Actor principal	Doctor o Paciente
Actores secundarios	AuthService y Router
Descripción	El usuario termina su sesión en la pestaña actual.
Precondiciones	Hay una sesión abierta en la pestaña.
Disparador	El usuario pulsa «Cerrar sesión».
Flujo principal	1. El usuario pulsa «Cerrar sesión» en el encabezado. 2. AuthService vacía la sesión en memoria. 3. AuthService borra la sesión de sessionStorage.

	4. El router navega a /login reemplazando la entrada actual del historial. 5. loginGuard no encuentra sesión y se muestra el login.
Flujos alternativos	A1. Cualquier respuesta 401 de la API ejecuta el mismo cierre de forma automática desde el interceptor. A2. Si el servidor está apagado, el cierre funciona igual porque es local.
Postcondiciones	No queda sesión en la pestaña. Ninguna pantalla protegida es accesible sin volver a ingresar.
Reglas de negocio	RN-02
Requisito relacionado	Perfiles por rol y seguridad de la información.
Interfaz	Ninguna.
Implementación	frontend: shared/encabezado.ts, core/auth.service.ts.

3.4 F-04: Consultar mis pacientes y su hilo

Al entrar, el doctor ve la lista de sus pacientes asignados y, a la derecha, el hilo del paciente elegido: lo que ya le envió y un formulario para escribirle una recomendación nueva. La pantalla se organiza por paciente porque los documentos del proyecto hablan de hilos de seguimiento por paciente.

La lista se obtiene de GET /api/doctor/pacientes, que devuelve solo los pacientes vinculados al doctor en la tabla doctor_paciente. Lo ya enviado se obtiene de GET /api/recomendaciones/emitidas. Ambas peticiones se hacen a la vez y las comparten las dos pestañas del panel (Pacientes y Resumen) mediante el servicio DoctorDatos, por lo que no se piden dos veces.

El hilo de cada paciente se arma en el navegador filtrando lo ya cargado, así que cambiar de paciente no genera peticiones. En pantallas anchas se abre el primer paciente automáticamente. En celular se muestra primero solo la lista y, al elegir un paciente, su hilo a pantalla completa con un botón para volver. Cambiar de paciente borra el borrador del formulario, para que un texto escrito para una persona no se envíe a otra por error.

Diagrama de secuencia

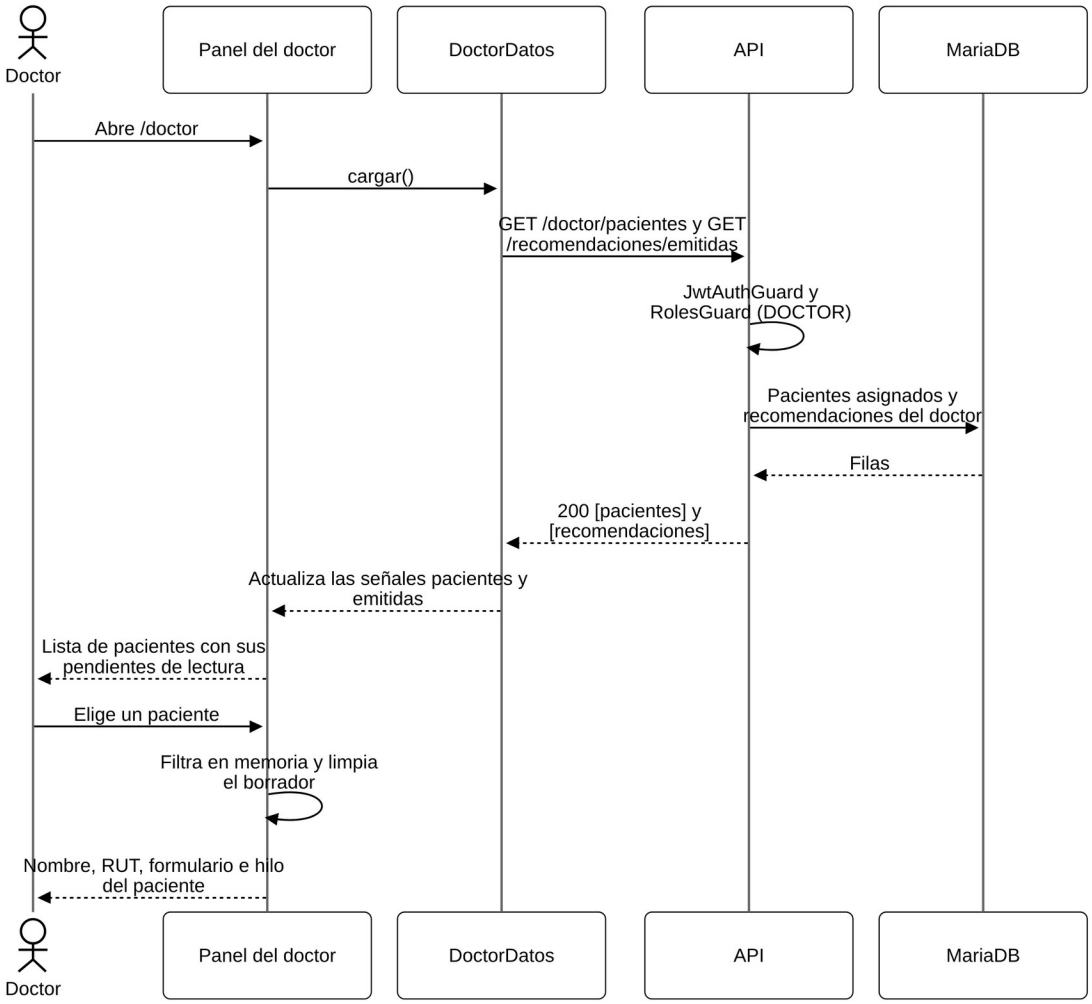


Figura 7. Secuencia del flujo F-04, Consultar mis pacientes y su hilo.

Caso de uso

Caso de uso CU-04: Consultar mis pacientes y su hilo	
Actor principal	Doctor
Actores secundarios	DoctorDatos, API y base de datos
Descripción	El doctor revisa sus pacientes asignados y lo que ya les envió.
Precondiciones	Hay una sesión de doctor verificada (flujo F-02).
Disparador	El doctor abre /doctor.
Flujo principal	1. El panel crea DoctorDatos, que pide en paralelo los pacientes y las recomendaciones emitidas.

	2. La API valida el token y el rol DOCTOR. 3. La API consulta los pacientes activos asignados al doctor y las recomendaciones que él envió. 4. El panel muestra la lista de pacientes con la cantidad de recomendaciones sin leer de cada uno. 5. El doctor elige un paciente. 6. La pantalla muestra su nombre, su RUT, el formulario y lo enviado, de la recomendación más nueva a la más antigua.
Flujos alternativos	A1. Doctor sin pacientes asignados: se muestra «Todavía no tiene pacientes asignados.» A2. Paciente sin recomendaciones: se muestra «Todavía no le ha enviado recomendaciones.» A3. Falla la carga: se muestra el error sobre la lista. A4. Un paciente desactivado no aparece en la lista.
Postcondiciones	Los datos quedan en memoria del panel. El doctor no accede a datos de pacientes que no le pertenecen.
Reglas de negocio	RN-04, RN-05
Requisito relacionado	Seguimiento de tratamientos y trazabilidad de acciones (Acta de constitución, OE3).
Interfaz	GET /api/doctor/pacientes. GET /api/recomendaciones/emitidas.
Implementación	frontend: doctor/doctor-datos.ts, doctor/pacientes-doctor.ts y .html. backend: asignaciones/doctor.controller.ts, recomendaciones/recomendaciones.service.ts.

3.5 F-05: Enviar una recomendación a un paciente

Es el flujo central del avance: el doctor escribe una indicación para uno de sus pacientes y el sistema la registra con su nombre, la fecha y la hora, de modo que queda disponible para el paciente.

El formulario pide el tipo (medicamento, rutina, hábito a evitar u otro), un título de 3 a 150 caracteres y las indicaciones, de 5 a 2000 caracteres. Antes de validar, el texto se recorta en los extremos y se eliminan los caracteres de control, por lo que un texto compuesto solo de espacios se rechaza tanto en el navegador como en el servidor.

El servidor toma el identificador del doctor desde el token y nunca desde el cuerpo de la solicitud, y comprueba que el paciente esté asignado a ese doctor. Si no lo está, responde 403 con el mismo mensaje tanto si el paciente existe como si no, para no revelar qué identificadores existen.

Mientras la solicitud está en curso el botón queda bloqueado y se ignoran los clics repetidos, para no enviar dos veces la misma recomendación. Si la solicitud falla, el borrador se conserva.

Diagrama de secuencia

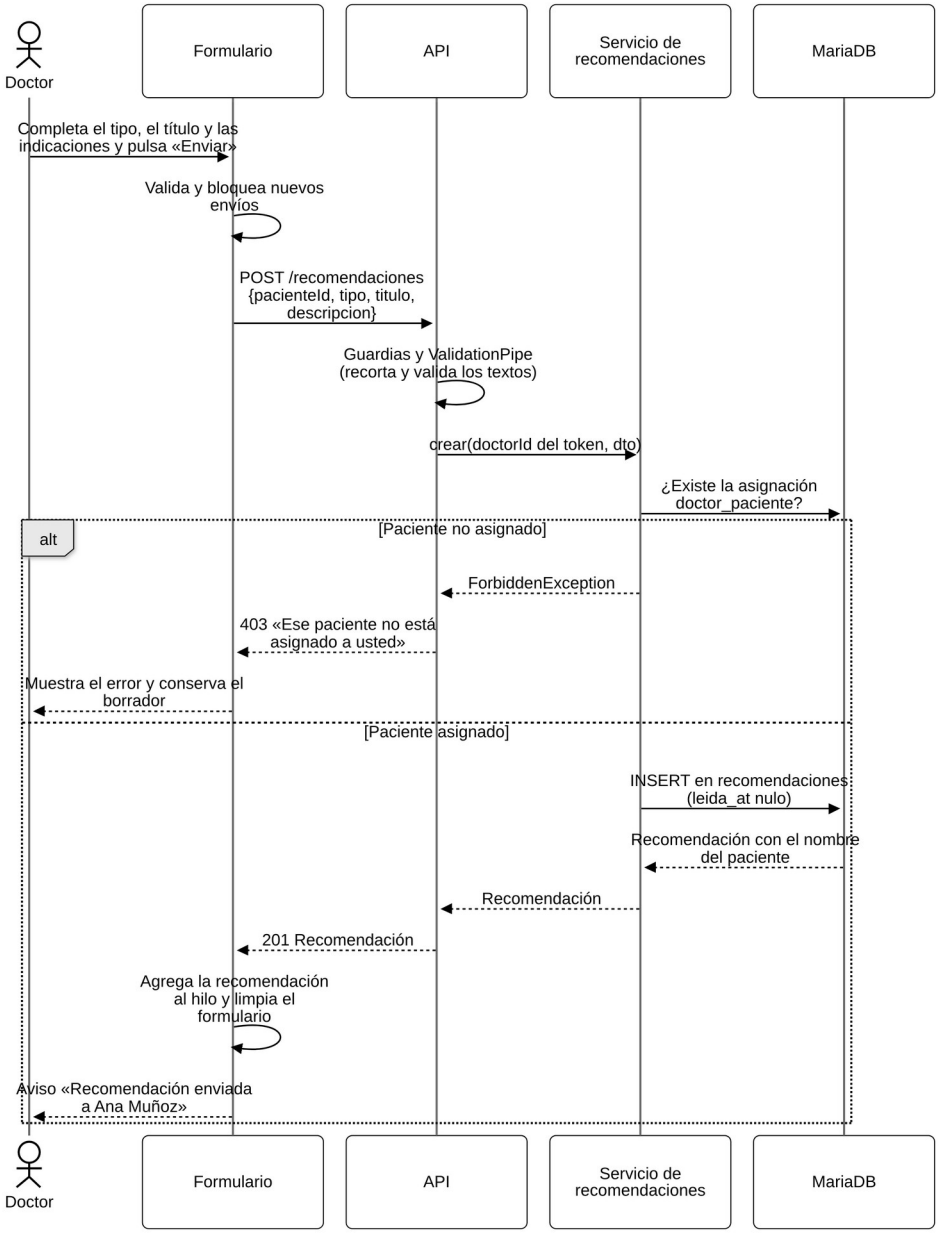


Figura 8. Secuencia del flujo F-05, Enviar una recomendación a un paciente.

Caso de uso

Caso de uso CU-05: Enviar una recomendación a un paciente	
Actor principal	Doctor
Actores secundarios	Paciente (destinatario), API y base de datos
Descripción	El doctor registra una recomendación dirigida a uno de sus pacientes.
Precondiciones	Hay una sesión de doctor. El paciente está asignado al doctor.
Disparador	El doctor pulsa «Enviar al paciente».
Flujo principal	1. El doctor elige un paciente (flujo F-04). 2. Selecciona el tipo de recomendación. 3. Escribe el título y las indicaciones. 4. Pulsa «Enviar al paciente». 5. La pantalla valida el formulario y bloquea nuevos envíos. 6. Envía POST /api/recomendaciones con pacienteId, tipo, titulo y descripcion. 7. La API valida el token y el rol, y recorta y valida los textos. 8. La API verifica que el paciente esté asignado al doctor del token. 9. Inserta la fila en recomendaciones con leida_at nulo. 10. Responde 201 con la recomendación creada. 11. La pantalla la agrega arriba del hilo, vacía el título y las indicaciones, conserva el tipo y muestra «Recomendación enviada a …».
Flujos alternativos	A1. Formulario inválido: se muestran los mensajes junto a cada campo y no se envía. A2. La API responde 400 por datos inválidos: se muestra el mensaje de validación sobre el botón. A3. Paciente no asignado o inexistente: responde 403 «Ese paciente no está asignado a usted». A4. Sesión vencida (401): se cierra la sesión y se vuelve a /login. A5. Sin conexión: se muestra el error y el borrador se conserva. A6. Doble clic: se ignoran los clics mientras hay una solicitud en curso.
Postcondiciones	Existe una nueva recomendación sin leer. El paciente la verá en su pantalla (flujo F-08).
Reglas de negocio	RN-04, RN-05, RN-08, RN-11
Requisito relacionado	Seguimiento de tratamientos: sugerencias del médico al paciente (OE3). Cubre las sugerencias; los recordatorios por WhatsApp o correo quedan pendientes.
Interfaz	POST /api/recomendaciones
Implementación	frontend: doctor/pacientes-doctor.ts y .html. backend: recomendaciones/recomendaciones.controller.ts, recomendaciones.service.ts, crear-recomendacion.dto.ts, common/texto.util.ts.

3.6 F-06: Ver el estado de lectura de lo enviado

El doctor necesita saber si el paciente leyó lo que le envió. Cada recomendación del hilo muestra «Aún sin leer» o «Leída el …», con la fecha y la hora en que el paciente la marcó.

Para que el estado se actualice sin recargar la página, DoctorDatos vuelve a pedir los datos cada 20 segundos mientras el panel está abierto. Estas actualizaciones son silenciosas: si una falla no se muestra un error y se conserva lo que ya estaba en pantalla, y el intento siguiente corrige la situación. El intervalo se cancela al salir del panel.

Se optó por consultar periódicamente y no por mantener una conexión permanente (WebSocket) porque el avance no lo requiere y una espera máxima de 20 segundos es aceptable para este uso.

Diagrama de secuencia

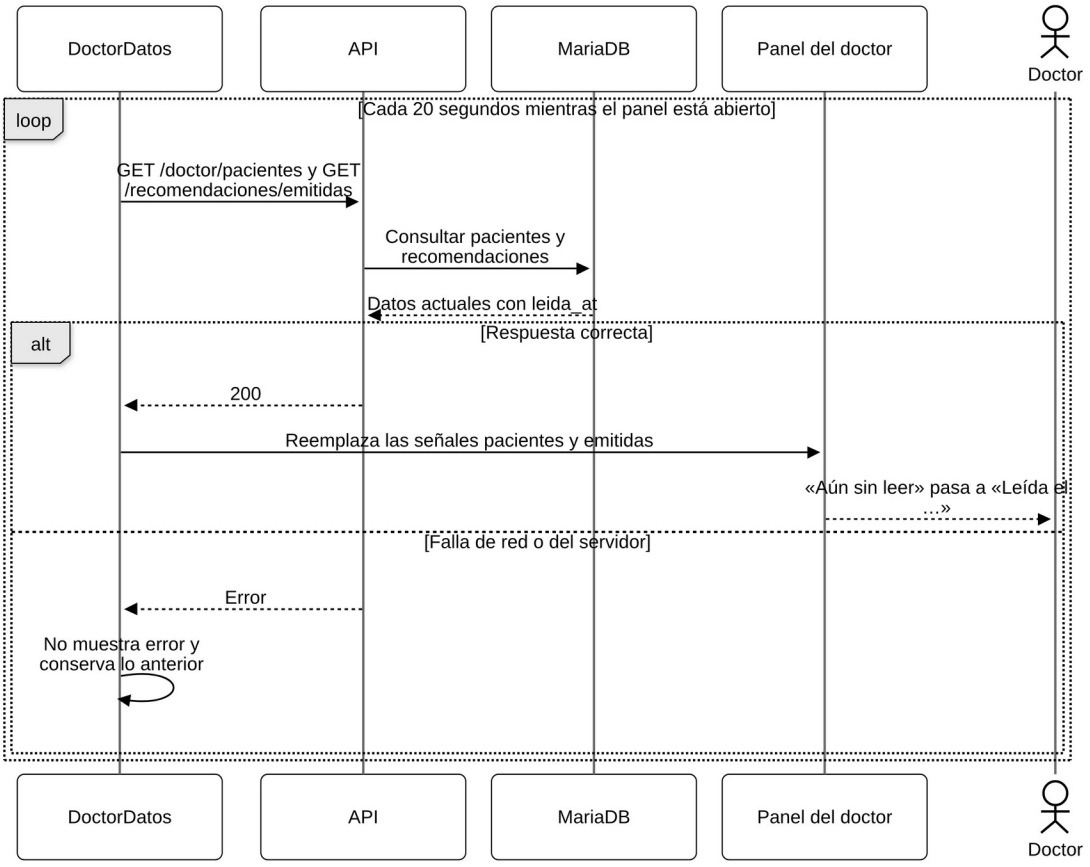


Figura 9. Secuencia del flujo F-06, Ver el estado de lectura con actualización automática.

Caso de uso

Caso de uso CU-06: Ver el estado de lectura de lo enviado	
Actor principal	Doctor
Actores secundarios	DoctorDatos, API y base de datos
Descripción	El doctor ve si cada recomendación fue leída, sin tener que recargar la página.
Precondiciones	El panel del doctor está abierto. Existen recomendaciones enviadas.
Disparador	Pasan 20 segundos desde la última carga.
Flujo principal	1. DoctorDatos ejecuta cargar en modo silencioso. 2. Pide los pacientes y las recomendaciones emitidas. 3. La API responde con los datos actuales, incluido leida_at. 4. DoctorDatos reemplaza los datos en memoria. 5. La pantalla muestra «Leída el …» y actualiza los contadores de sin leer.
Flujos alternativos	A1. Falla de red o del servidor: no se muestra un error y se mantiene lo anterior. A2. Sesión vencida (401): el interceptor cierra la sesión. A3. El doctor sale del panel: el intervalo se cancela.
Postcondiciones	La pantalla refleja la base de datos con un desfase máximo cercano a 20 segundos.
Reglas de negocio	RN-06
Requisito relacionado	Trazabilidad de acciones y estado de cada hilo (OE3 y OE5, de forma parcial).
Interfaz	GET /api/doctor/pacientes. GET /api/recomendaciones/emitidas.
Implementación	frontend: doctor/doctor-datos.ts.

3.7 F-07: Ver el resumen de lectura

La pestaña Resumen responde dos preguntas del doctor: si sus indicaciones se están leyendo y a quién le quedan pendientes. No contiene cifras de ejemplo: todo se calcula con las recomendaciones que ese doctor realmente envió y que ya están cargadas en memoria, por lo que abrir la pestaña no genera una petición al servidor.

Muestra una frase con los totales (por ejemplo, «Ha enviado 4 recomendaciones a 2 pacientes. 2 fueron leídas y siguen 2 sin leer.»), un gráfico de barras con las recomendaciones leídas y sin leer por paciente, un gráfico por tipo y la lista de pendientes ordenada desde la más antigua, con el tiempo transcurrido («hoy», «ayer», «hace N

días»). Cada gráfico tiene una tabla equivalente, oculta a la vista, para lectores de pantalla.

Este resumen cubre solo lo que el sistema registra hoy. Un dashboard de continuidad con hilos abiertos, finalizados y desertores, como lo define el Plan de Gestión de Beneficios, necesita el estado de seguimiento de cada paciente, que se construirá en una etapa posterior.

Diagrama de secuencia

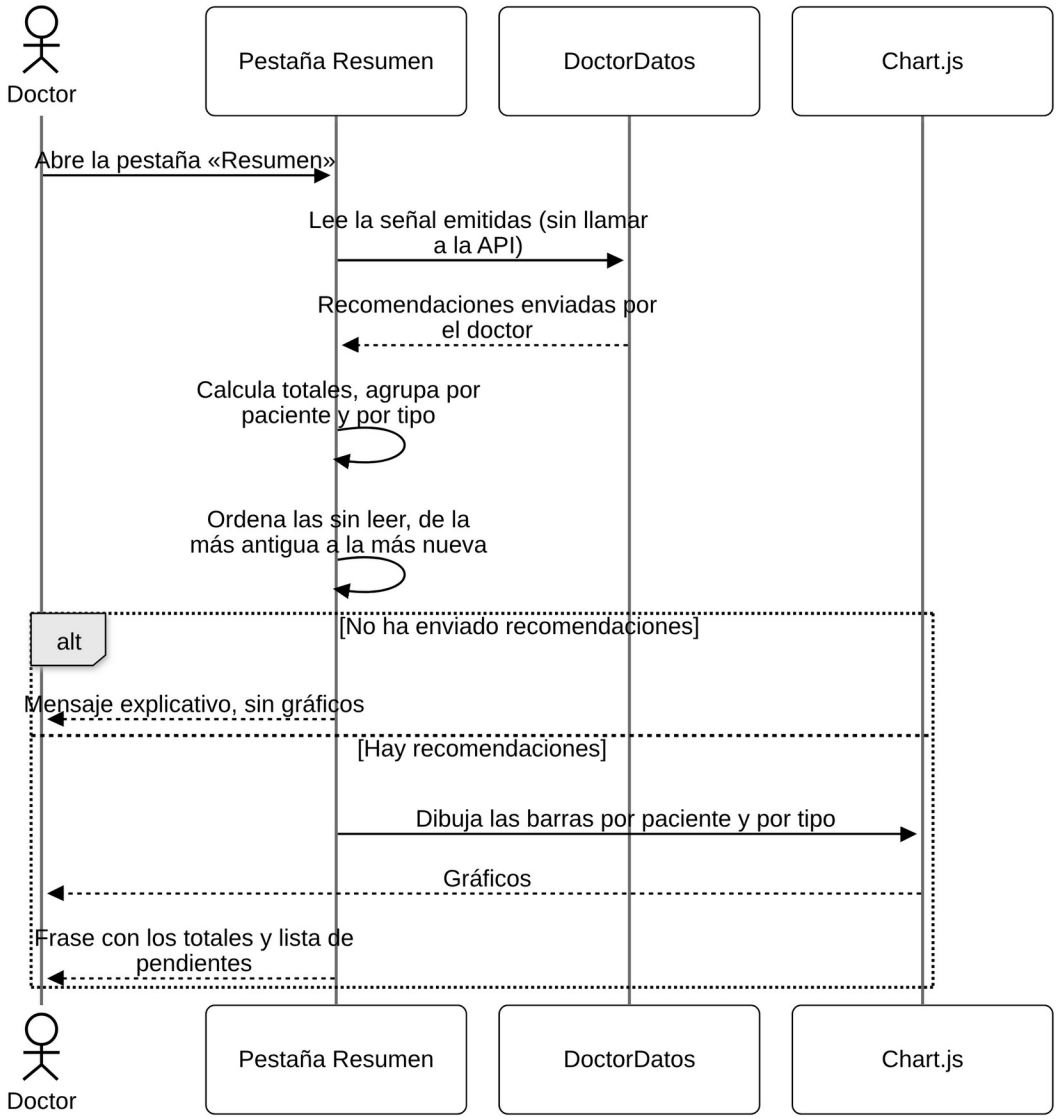


Figura 10. Secuencia del flujo F-07, Ver el resumen de lectura.

Caso de uso

Caso de uso CU-07: Ver el resumen de lectura	
Actor principal	Doctor
Actores secundarios	DoctorDatos y Chart.js
Descripción	El doctor revisa cuántas de sus recomendaciones se leen y a quién le quedan pendientes.
Precondiciones	Hay una sesión de doctor. DoctorDatos cargó las recomendaciones emitidas.
Disparador	El doctor abre la pestaña «Resumen».
Flujo principal	1. El doctor abre la pestaña «Resumen». 2. La pantalla lee las recomendaciones emitidas desde DoctorDatos. 3. Calcula el total, las leídas y las sin leer, y las agrupa por paciente y por tipo. 4. Ordena las sin leer de la más antigua a la más nueva y toma las ocho primeras. 5. Dibuja los gráficos con Chart.js, la frase de totales y la lista de pendientes.
Flujos alternativos	A1. El doctor no ha enviado recomendaciones: se muestra un mensaje que explica qué verá cuando envíe la primera, sin gráficos. A2. No hay recomendaciones sin leer: se muestra «No hay recomendaciones sin leer. Sus pacientes están al día.» A3. Cuando DoctorDatos se actualiza, el resumen se recalcula solo.
Postcondiciones	No se modifica ningún dato.
Reglas de negocio	RN-06
Requisito relacionado	Dashboard de continuidad para médicos (OE5). Cubre el resumen de lectura; faltan los indicadores de continuidad y de desertores.
Interfaz	Ninguna propia. Usa los datos de los flujos F-04 y F-06.
Implementación	frontend: doctor/resumen-doctor.ts, .html y .css.

3.8 F-08: Ver mis recomendaciones

El paciente ve todas las recomendaciones que le enviaron sus doctores, de cualquiera de ellos, ordenadas con las no leídas primero y, dentro de cada grupo, de la más nueva a la más antigua. Arriba se resume su situación con una frase directa («Tiene 2 recomendaciones sin leer» o «Está al día: no tiene recomendaciones sin leer»). Cada recomendación muestra su tipo, título e indicaciones, el nombre y la especialidad del doctor y la fecha de envío.

El paciente nunca indica su propio identificador: la API lo toma del token, de modo que no existe una forma de pedir las recomendaciones de otra persona. La pantalla se actualiza sola cada 20 segundos para mostrar las nuevas sin recargar. La tipografía y el tamaño de letra se eligieron pensando en pacientes de edad avanzada.

Diagrama de secuencia

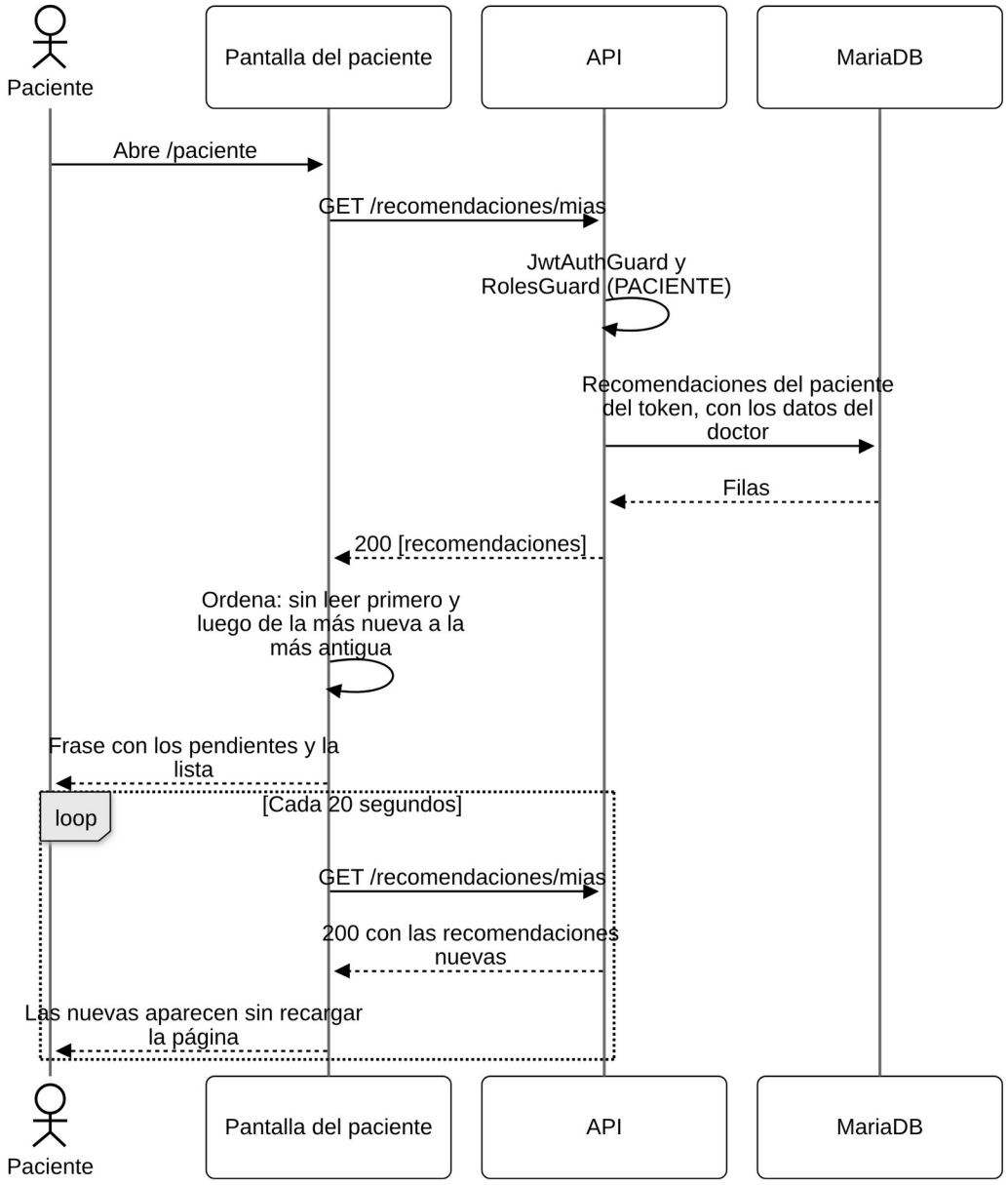


Figura 11. Secuencia del flujo F-08, Ver mis recomendaciones.

Caso de uso

Caso de uso CU-08: Ver mis recomendaciones	
Actor principal	Paciente
Actores secundarios	API y base de datos
Descripción	El paciente revisa las recomendaciones que recibió de sus doctores.
Precondiciones	Hay una sesión de paciente verificada (flujo F-02).
Disparador	El paciente abre /paciente o inicia sesión.
Flujo principal	1. La pantalla envía GET /api/recomendaciones/mias. 2. La API valida el token y el rol PACIENTE. 3. La API consulta las recomendaciones del paciente del token, con los datos del doctor. 4. La pantalla las ordena (sin leer primero) y muestra la frase de resumen. 5. Cada 20 segundos repite la consulta sin mostrar un indicador de carga.
Flujos alternativos	A1. Sin recomendaciones: se muestra «Todavía no tiene recomendaciones. Cuando un doctor le envíe una, aparecerá aquí.» A2. Falla la consulta inicial: se muestra un aviso en la parte superior. A3. Sesión vencida (401): se cierra la sesión.
Postcondiciones	No se modifica ningún dato.
Reglas de negocio	RN-03, RN-05, RN-11
Requisito relacionado	Seguimiento de tratamientos: el paciente recibe recomendaciones personalizadas (Plan de Gestión de Beneficios, B5).
Interfaz	GET /api/recomendaciones/mias
Implementación	frontend: paciente/paciente.ts, .html y .css. backend: recomendaciones/recomendaciones.controller.ts y recomendaciones.service.ts.

3.9 F-09: Marcar una recomendación como leída

El paciente confirma que leyó una recomendación con el botón «Marcar como leída». Es la forma en que el sistema deja constancia de que la indicación llegó a la persona, lo que aporta a la trazabilidad que pide el centro.

La API busca la recomendación con el identificador indicado y el identificador del paciente del token. Si la recomendación no existe o pertenece a otro paciente, responde 404 con el mismo mensaje, de modo que no se puede averiguar si el identificador existe. La

operación es idempotente: si ya estaba leída conserva la fecha original, así que pulsar el botón varias veces o repetir la solicitud no la modifica. La marca no se puede deshacer.

En pantalla, los clics repetidos sobre la misma recomendación se ignoran mientras hay una solicitud pendiente. Cuando la API responde, la recomendación pasa a «Leída el …», baja el contador de pendientes y aparece el aviso «Marcada como leída». El doctor verá el cambio en su pantalla en un máximo de 20 segundos (flujo F-06).

Diagrama de secuencia

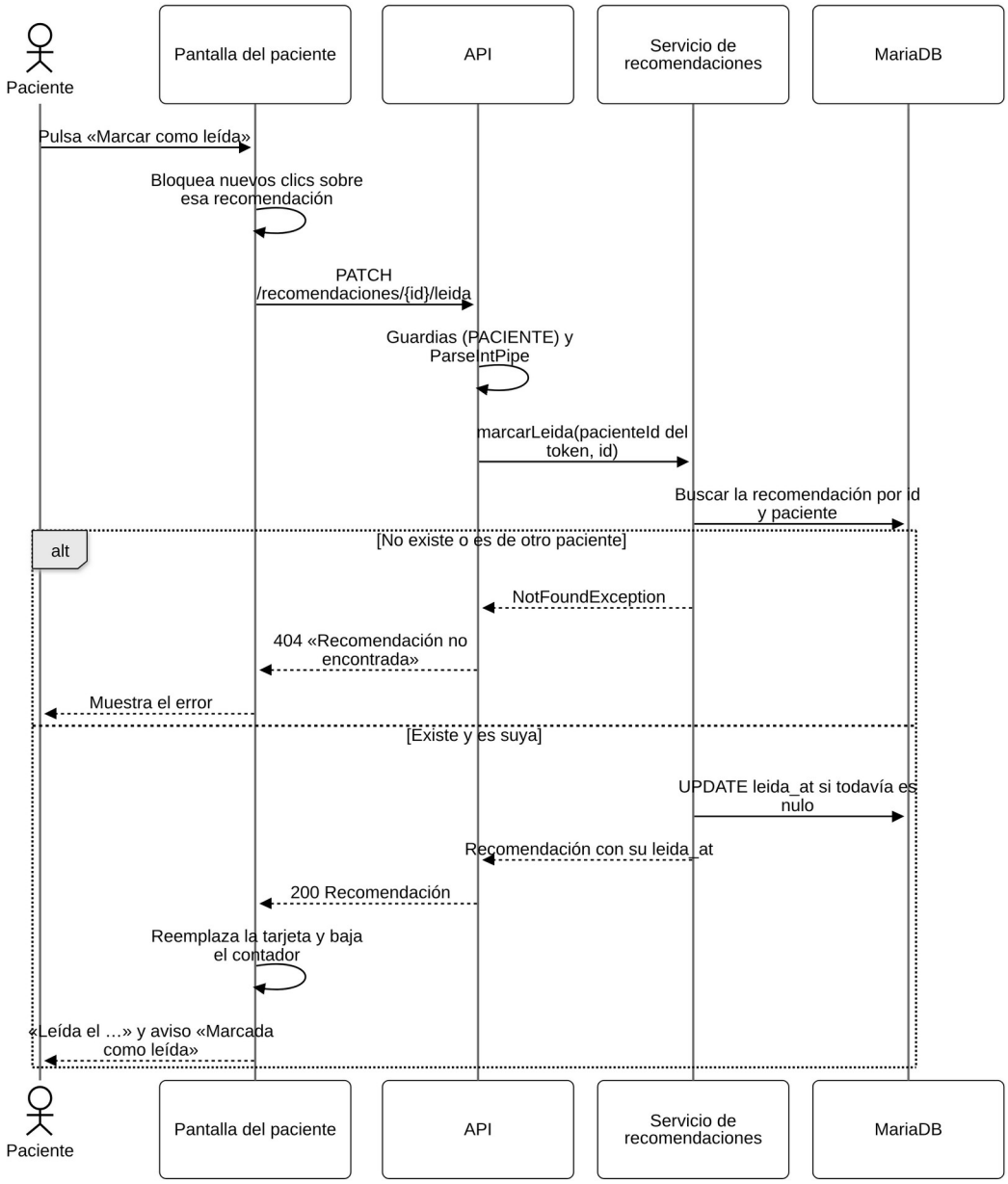


Figura 12. Secuencia del flujo F-09, Marcar una recomendación como leída.

La figura siguiente resume los dos estados posibles de una recomendación y lo que provoca el cambio entre ellos.

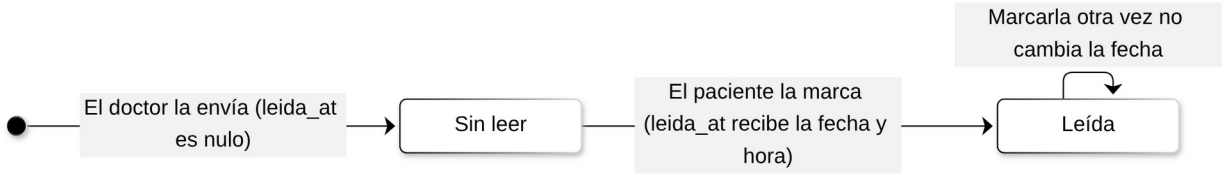


Figura 13. Estados de una recomendación.

Caso de uso

Caso de uso CU-09: Marcar una recomendación como leída	
Actor principal	Paciente
Actores secundarios	Doctor (verá el cambio), API y base de datos
Descripción	El paciente deja registrado que leyó una recomendación.
Precondiciones	Hay una sesión de paciente. La recomendación está dirigida a ese paciente.
Disparador	El paciente pulsa «Marcar como leída».
Flujo principal	1. El paciente pulsa «Marcar como leída» en una recomendación. 2. La pantalla bloquea nuevos clics sobre esa recomendación. 3. Envía PATCH /api/recomendaciones/{id}/leida. 4. La API valida el token, el rol PACIENTE y que el id sea un número entero. 5. Busca la recomendación por id y por paciente. 6. Si leida_at es nulo, guarda la fecha y la hora actuales. 7. Responde 200 con la recomendación actualizada. 8. La pantalla reemplaza la recomendación, baja el contador y muestra el aviso.
Flujos alternativos	A1. El id no es un número entero: responde 400. A2. La recomendación no existe o es de otro paciente: responde 404 «Recomendación no encontrada». A3. Ya estaba leída: responde 200 con la fecha original, sin modificarla. A4. Falla de red: se muestra el error y la recomendación sigue como sin leer. A5. Un doctor intenta usar la ruta: responde 403.
Postcondiciones	leida_at queda registrada una sola vez. El doctor verá la recomendación como leída en su próxima actualización.
Reglas de negocio	RN-05, RN-06, RN-07, RN-11
Requisito relacionado	Trazabilidad de todas las interacciones con el paciente (Plan de Gestión de Beneficios, B2).
Interfaz	PATCH /api/recomendaciones/:id/leida

Implementación	frontend: paciente/paciente.ts y .html. backend: recomendaciones/recomendaciones.controller.ts y recomendaciones.service.ts (marcarLeida).

3.10 F-10: Autorizar cada petición a la API

Toda petición a la API, salvo el login, pasa por dos guardias globales antes de llegar al controlador. La primera, JwtAuthGuard, extrae el token del encabezado Authorization (el esquema Bearer se acepta sin distinguir mayúsculas), comprueba la firma y el vencimiento y luego consulta en la base de datos que el usuario siga existiendo y activo. El rol y el nombre que se usan en el resto de la petición salen de la base de datos y no del token. Así, un usuario desactivado pierde el acceso de inmediato y un cambio de rol se aplica sin esperar a que el token venza.

La segunda guardia, RolesGuard, compara el rol del usuario con el que exige la ruta (decorador @Roles) y responde 403 si no coincide.

En el navegador, un interceptor HTTP agrega el token a las llamadas a /api y, si cualquier respuesta es 401, cierra la sesión y envía al usuario a /login. Este flujo no lo inicia un actor: ocurre dentro de todos los demás, y por eso se documenta aparte.

Diagrama de secuencia

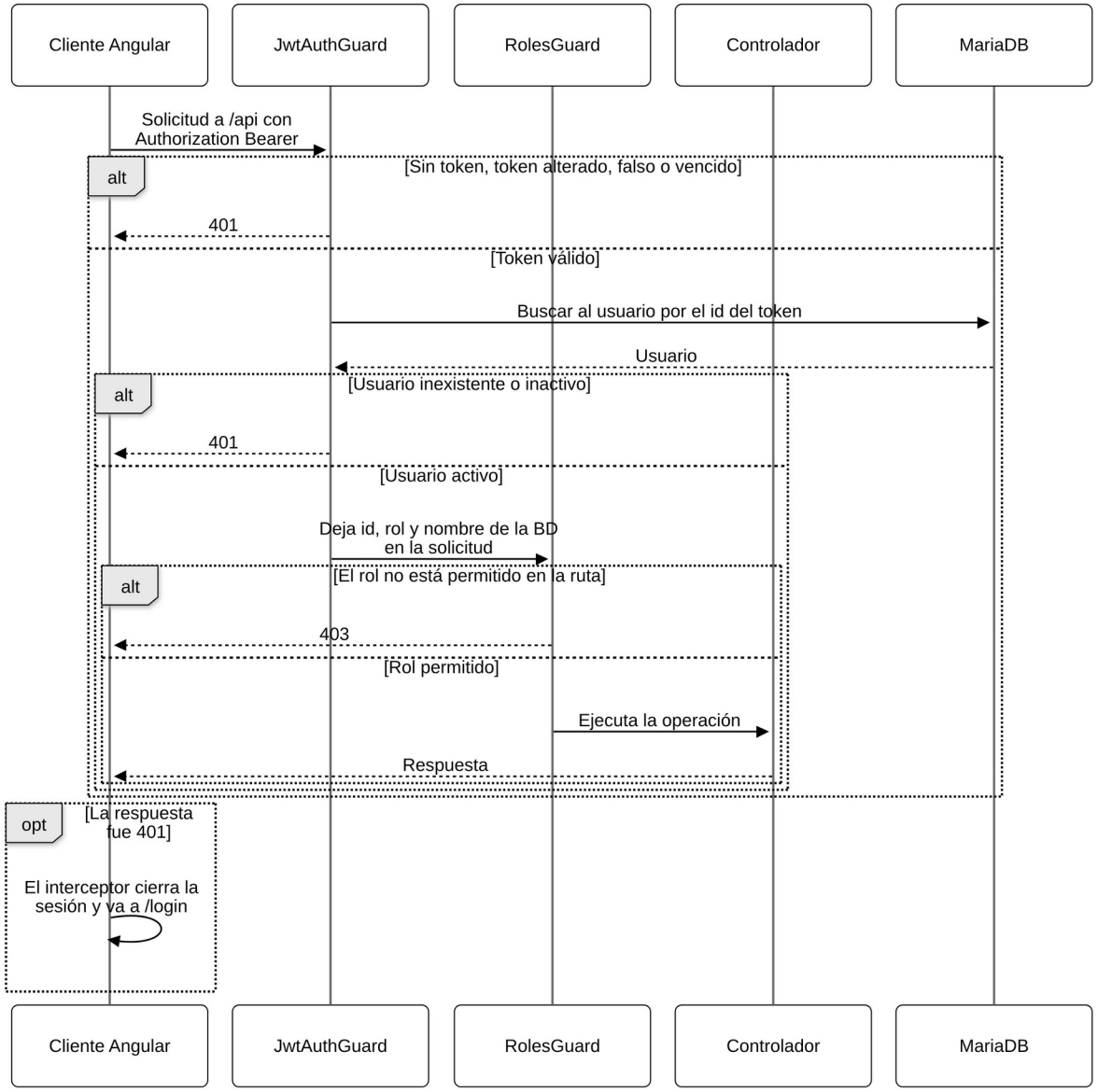


Figura 14. Secuencia del flujo F-10, Autorización de cada petición a la API.

Caso de uso

Caso de uso CU-10: Autorizar cada petición a la API	
Actor principal	Sistema (en nombre de cualquier usuario autenticado)
Actores secundarios	Interceptor HTTP, JwtAuthGuard, RolesGuard y base de datos
Descripción	Comprobar en cada petición quién la hace y si tiene permiso para esa operación.

Precondiciones	Ninguna.
Disparador	Llega cualquier solicitud a /api.
Flujo principal	1. El interceptor agrega Authorization: Bearer con el token, salvo en /auth/login. 2. JwtAuthGuard comprueba si la ruta es pública. 3. Extrae el token y verifica su firma y su vencimiento. 4. Consulta al usuario en la base de datos: debe existir y estar activo. 5. Deja el id, el rol y el nombre de la base de datos en la solicitud. 6. RolesGuard compara el rol con el que exige la ruta. 7. Si coincide, se ejecuta el controlador y se devuelve la respuesta.
Flujos alternativos	A1. Sin encabezado, con un esquema distinto de Bearer o sin token: responde 401 «Debe iniciar sesión». A2. Token con firma falsa, algoritmo none, alterado o vencido: responde 401 «Sesión inválida o expirada». A3. Usuario inexistente o inactivo: responde 401 «Sesión inválida o expirada». A4. Rol no permitido en la ruta: responde 403 «No tiene permisos para esta acción». A5. Ante cualquier 401 el interceptor cierra la sesión y envía a /login.
Postcondiciones	El controlador solo se ejecuta con un usuario válido y autorizado.
Reglas de negocio	RN-03, RN-09, RN-11
Requisito relacionado	Perfiles por rol y máxima seguridad posible de la información (OE4).
Interfaz	Todas las rutas de /api.
Implementación	backend: auth/jwt-auth.guard.ts, auth/roles.guard.ts, auth/decorators.ts. frontend: core/auth.interceptor.ts.

3.11 F-11: Aplicar migraciones y cargar datos de demostración

El esquema de la base de datos cambia solo mediante migraciones versionadas, nunca con la sincronización automática de TypeORM (synchronize está desactivado). Cada migración tiene un método up(), que aplica el cambio, y un método down(), que lo deshace. Hay tres migraciones: usuarios, doctor_paciente y recomendaciones. TypeORM registra las ejecutadas en la tabla migrations y aplica solo las pendientes.

El seed carga los usuarios de demostración (dos doctores y tres pacientes), sus asignaciones y ocho recomendaciones de ejemplo con fechas escalonadas. Es repetible: busca cada registro antes de insertarlo y no duplica. Con la opción --reset borra las recomendaciones y las vuelve a crear, para dejar la demostración como nueva.

Las fechas se guardan en UTC (valor por defecto UTC_TIMESTAMP y zona horaria Z en la conexión), para que no dependan de la zona horaria del servidor.

Diagrama de secuencia

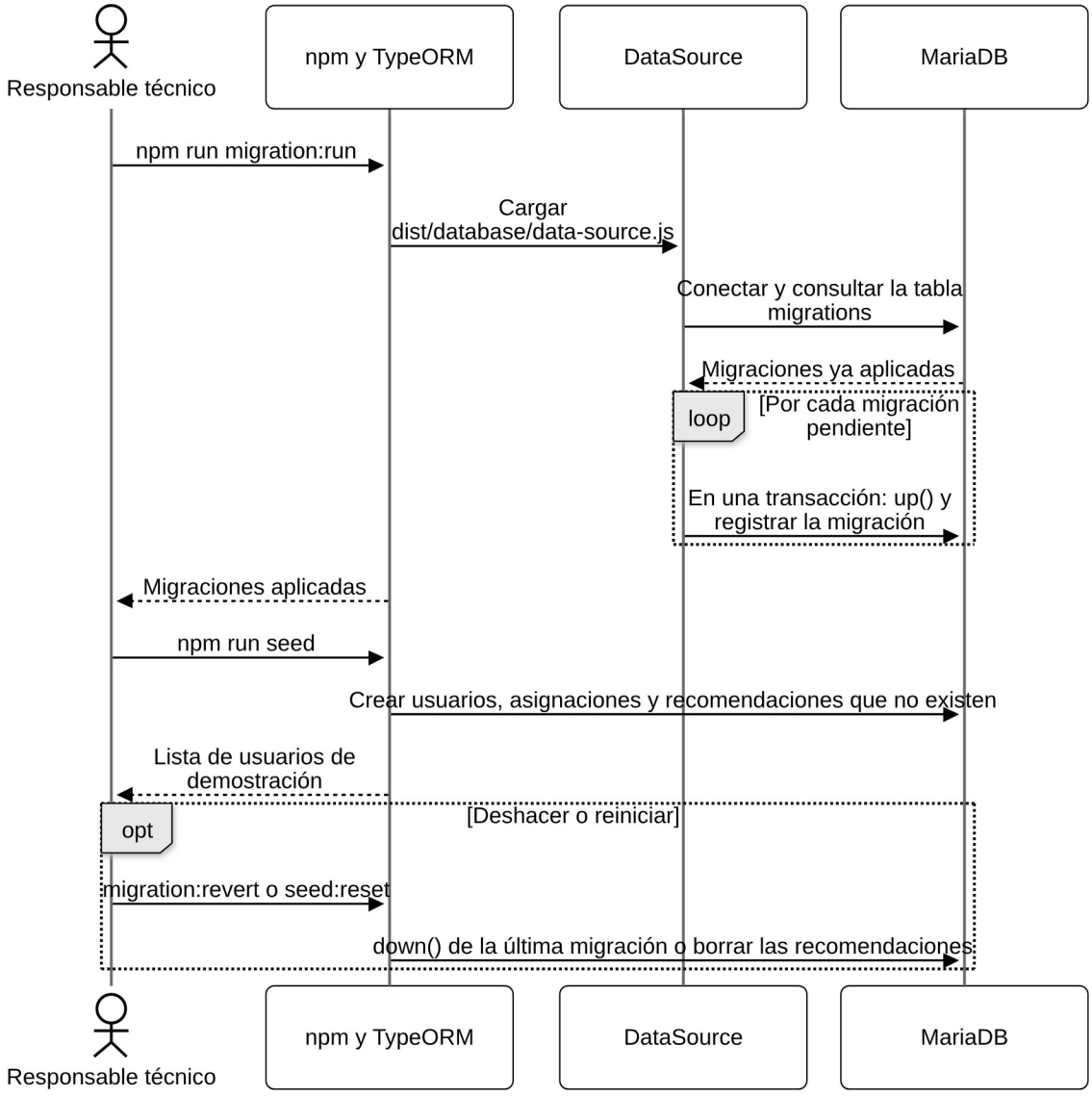


Figura 15. Secuencia del flujo F-11, Migraciones de la base de datos y datos de demostración.

Caso de uso

Caso de uso CU-11: Aplicar migraciones y cargar datos de demostración	
Actor principal	Responsable técnico del equipo

Actores secundarios	TypeORM y MariaDB
Descripción	Crear o actualizar las tablas y dejar la base de datos con datos para la demostración.
Precondiciones	La base de datos vida_norte y el usuario arm_app existen (ver el README). El archivo .env está completo.
Disparador	El responsable ejecuta npm run migration:run y luego npm run seed.
Flujo principal	1. npm compila el backend (nest build). 2. TypeORM carga dist/database/data-source.js, que lee el .env y se conecta a MariaDB. 3. Consulta la tabla migrations para saber cuáles ya se aplicaron. 4. Por cada migración pendiente, en orden, ejecuta up() dentro de una transacción y la registra. 5. npm run seed calcula el hash bcrypt de la contraseña de demostración. 6. Crea los usuarios que no existen, buscándolos por RUT. 7. Crea las asignaciones doctor_paciente que faltan. 8. Crea las recomendaciones de ejemplo que no existen. 9. Muestra la lista de usuarios de demostración.
Flujos alternativos	A1. Deshacer: npm run migration:revert ejecuta down() de la última migración. A2. Reiniciar la demostración: npm run seed:reset borra las recomendaciones y las vuelve a crear. A3. Falla la conexión: se informa el error y no se aplica ningún cambio. A4. Seed repetido: no duplica usuarios, asignaciones ni recomendaciones.
Postcondiciones	Las tres tablas existen. La base de datos contiene 5 usuarios, 4 asignaciones y 8 recomendaciones de ejemplo.
Reglas de negocio	RN-07
Requisito relacionado	Migraciones de la base de datos solicitadas para el avance.
Interfaz	Línea de comandos: migration:run, migration:revert, migration:show, seed y seed:reset.
Implementación	backend: database/data-source.ts, database/database.config.ts, database/migrations/ y database/seed.ts.

4. Manejo de errores

La API responde con el código HTTP que corresponde y la pantalla traduce cada caso a un mensaje en español que explica qué ocurrió y qué hacer. Los errores del servidor no exponen detalles internos.

Código	Cuándo ocurre	Qué ve el usuario
400	Datos inválidos: RUT con dígito incorrecto, textos muy cortos o largos, tipo inexistente, id no numérico.	Mensaje en español junto a los campos o sobre el botón.
401	Sin sesión, token inválido o vencido, usuario inexistente o inactivo, credenciales incorrectas en el login.	En el login, mensaje bajo el formulario. En el resto, se cierra la sesión y se vuelve a /login.
403	Rol no permitido en la ruta, o paciente no asignado al doctor.	Mensaje «No tiene permisos para esta acción» o «Ese paciente no está asignado a usted».
404	Recomendación inexistente o de otro paciente.	Mensaje «Recomendación no encontrada».
0, 502, 503, 504	Sin conexión o servidor no disponible.	Mensaje «No se pudo conectar con el servidor…». Los datos ya cargados siguen a la vista.
500	Error interno del servidor.	Mensaje genérico, sin detalles internos.

5. Limitaciones conocidas

Estas limitaciones están identificadas y quedan para las etapas siguientes del proyecto.

1. No hay límite de intentos de inicio de sesión, por lo que un atacante puede probar contraseñas sin freno. Conviene agregar @nestjs/throttler antes de usar el sistema con datos reales.

2. La API no envía cabeceras de seguridad (por ejemplo, con helmet).

3. El cierre de sesión es local. El token sigue siendo válido hasta que venza, aunque un usuario desactivado pierde el acceso en la siguiente petición (RN-09).

4. Las recomendaciones no se pueden editar ni eliminar, y la marca de lectura no se puede deshacer.

5. La actualización de las pantallas es por consulta cada 20 segundos, no en tiempo real.

6. Las listas no tienen paginación: el doctor carga todas las recomendaciones que envió.

7. No existe historial de accesos, cambio ni recuperación de contraseña, ni Clave Única simulada.

8. Todos los usuarios de demostración comparten una misma contraseña, definida en el seed. No debe usarse fuera de pruebas.

6. Evidencia del incremento

6.1 Módulos funcionales entregados

Módulo	Flujos	Estado
Autenticación y perfiles por rol	F-01, F-02, F-03 y F-10	Funcional. Dos roles (doctor y paciente) con autorización en cada petición.
Recomendaciones y seguimiento de lectura	F-04, F-05, F-06, F-08 y F-09	Funcional. El doctor envía a sus pacientes y el paciente confirma la lectura.
Resumen para el doctor	F-07	Funcional con datos reales. Cubre la lectura de las recomendaciones, no la continuidad completa.
Base de datos y migraciones	F-11	Funcional. Tres migraciones reversibles y datos de demostración.

6.2 Pruebas realizadas

Tipo de prueba	Contenido	Resultado
Pruebas unitarias del backend (Vitest)	9 pruebas: validación de RUT y limpieza de texto.	Aprobadas
Pruebas unitarias del frontend (Vitest)	29 pruebas: RUT, sesión por pestaña, verificación con el servidor, validadores, fechas y mensajes de error.	Aprobadas
Batería de la API con entradas hostiles (qa/api_hostil.py)	69 casos: login, tokens falsos o vencidos, límites de campos, ids extraños y CORS.	Aprobada
Recorrido en el navegador	Flujo completo entre un doctor y un paciente en dos pestañas, rutas directas, botón Atrás, doble clic, servidor apagado y diseño en celular.	Realizado

6.3 Constancia de revisión y aprobación

Espacio para dejar constancia de que el incremento fue presentado y revisado por el cliente o el docente.

Revisado por	Cargo	Fecha	Firma

Anexo A. Rutas de la API

Método	Ruta	Rol	Descripción
POST	/api/auth/login	Público	Inicia sesión con RUT y contraseña. Devuelve el token y los datos del usuario.
GET	/api/auth/me	Cualquiera	Devuelve id, rol y nombre del usuario según la base de datos.
GET	/api/doctor/pacientes	DOCTOR	Lista los pacientes asignados al doctor.
POST	/api/recomendaciones	DOCTOR	Crea una recomendación para un paciente asignado.
GET	/api/recomendaciones/emitidas	DOCTOR	Lista las recomendaciones que el doctor envió, con su estado de lectura.
GET	/api/recomendaciones/mias	PACIENTE	Lista las recomendaciones que el paciente recibió.
PATCH	/api/recomendaciones/:id/leida	PACIENTE	Marca una recomendación propia como leída.

Anexo B. Mapa del código

Carpeta	Responsabilidad
backend/src/auth/	Inicio de sesión, token JWT y guardias globales de autenticación y de roles.
backend/src/asignaciones/	Vínculo entre doctores y pacientes y listado de pacientes del doctor.
backend/src/recomendaciones/	Creación, consulta y marca de lectura de las recomendaciones.
backend/src/usuarios/	Entidad Usuario y roles.

Carpeta	Responsabilidad
backend/src/common/	Validación del RUT y limpieza de texto.
backend/src/database/	Configuración de la conexión, migraciones y seed.
frontend/src/app/core/	Sesión, guardias del router, interceptor HTTP, servicios de datos, validadores y utilidades.
frontend/src/app/login/	Pantalla de inicio de sesión.
frontend/src/app/doctor/	Panel del doctor: pacientes, hilo, redactor y resumen.
frontend/src/app/paciente/	Pantalla del paciente.
frontend/src/app/shared/	Encabezado y logo.
qa/	Batería de pruebas de la API con entradas hostiles.